

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical - IX

Roll No.: T028	Name: Akash Negi
Class: TYIT	Batch: 1
Date of Assignment: 18-08-2026	Date/Time of Submission: 18-08-2026

PART B – Questions

- a. **Develop CRUD operations using ASP.NET Core Web API and Entity Framework Core.**

Code:

Student.cs

namespace StudentAPI.Models

```
{  
    public class Student  
    {  
        public int ID { get; set; }  
        public string Name { get; set; }  
        public int Age { get; set; }  
        public string Course { get; set; }  
    }  
}
```

AppDbContext.cs

using Microsoft.EntityFrameworkCore;

using StudentAPI.Models;

namespace StudentAPI.Data

```
{  
    public class AppDbContext : DbContext  
    {  
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)  
        {  
        }  
  
        public DbSet<Student> Students { get; set; }  
    }  
}
```

appsetting.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"server=(localdb)\\mssqllocaldb;database=StudentDB;trusted_connection=true;"
  }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using StudentAPI.Data;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();

builder.Services.AddDbContext<AppDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

StudentsController.cs

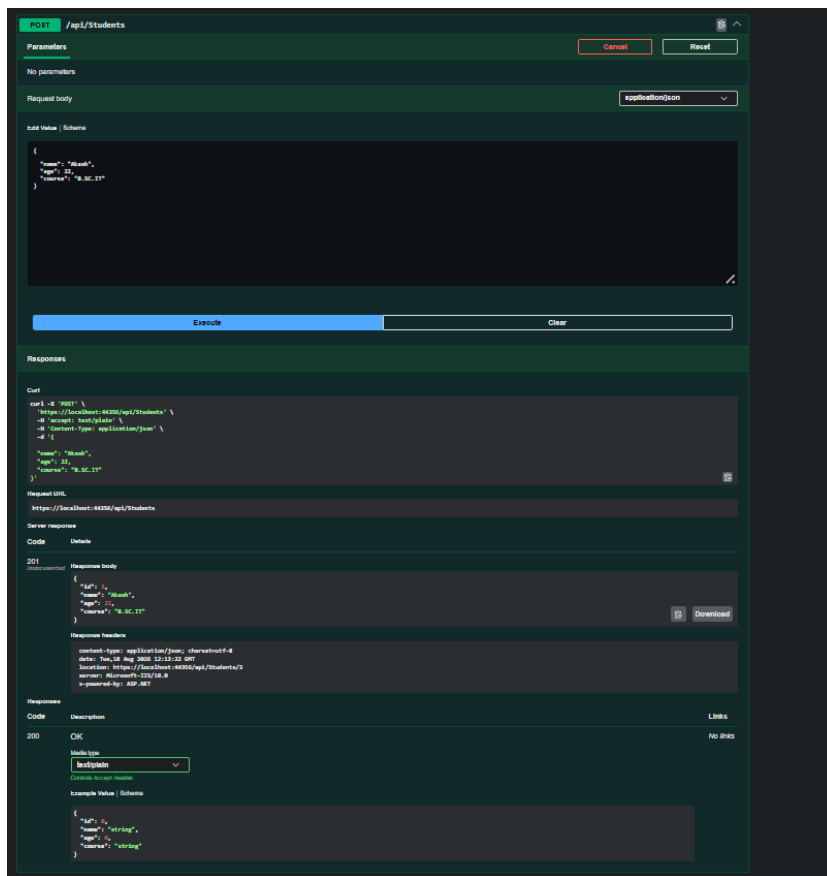
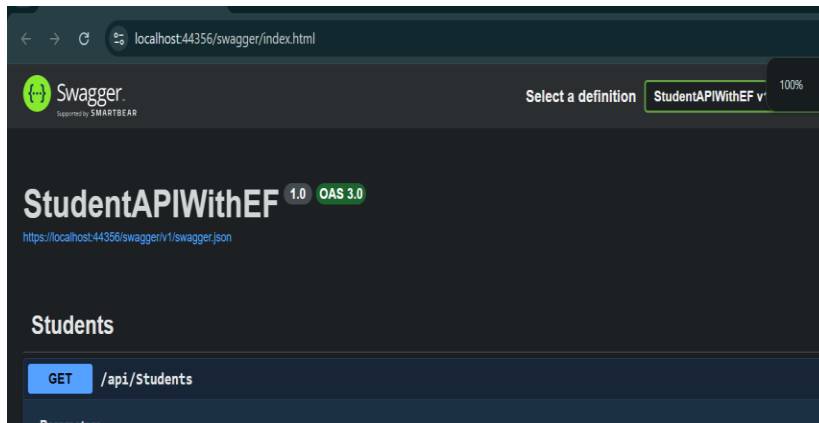
```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using StudentAPI.Data;
```

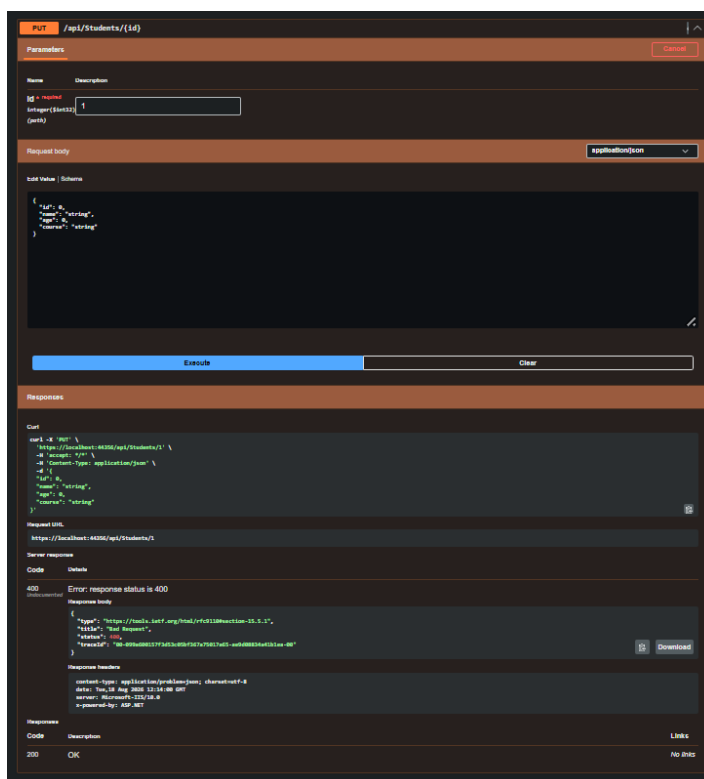
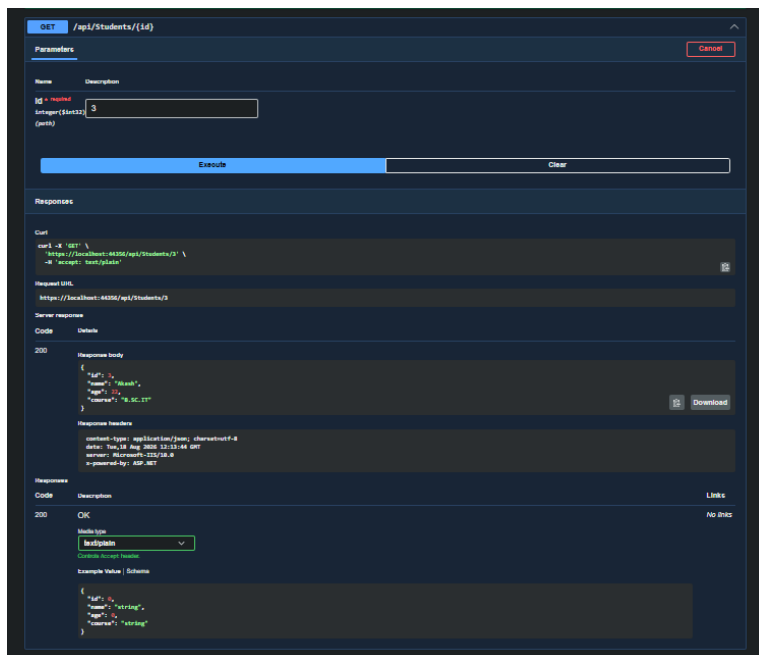
```
using StudentAPI.Models;

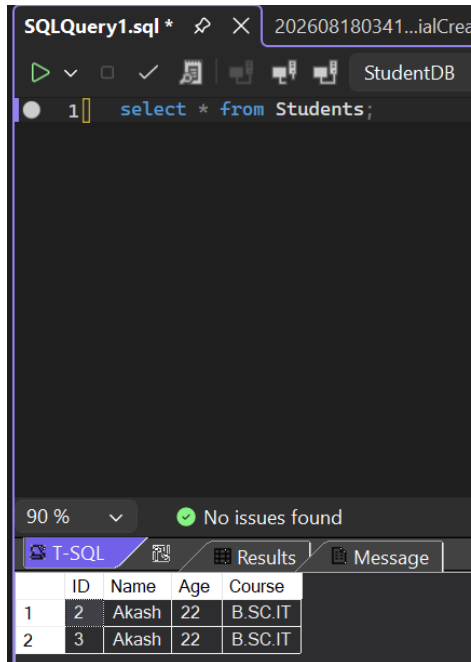
namespace StudentAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly AppDbContext _context;
        public StudentsController(AppDbContext context)
        {
            _context = context;
        }
        // GET: api/Student
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Student>>> GetStudents()
        {
            return await _context.Students.ToListAsync();
        }
        // GET: api/Student/5
        [HttpGet("{id}")]
        public async Task<ActionResult<Student>> GetStudent(int id)
        {
            var student = await _context.Students.FindAsync(id);
            if (student == null)
            {
                return NotFound();
            }
            return student;
        }
        // POST: api/Student
        [HttpPost]
        public async Task<ActionResult<Student>> PostStudent(Student student)
        {
            _context.Students.Add(student);
            await _context.SaveChangesAsync();
            return CreatedAtAction(nameof(GetStudent), new { id = student.ID }, student);
        }
        // PUT: api/Student/5
        [HttpPut("{id}")]
        public async Task<ActionResult> PutStudent(int id, Student student)
        {
            if (id != student.ID)
            {
```

```
        return BadRequest();
    }
    _context.Entry(student).State = EntityState.Modified;
    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!StudentExists(id))
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }
    return NoContent();
}
// DELETE: api/Student/5
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteStudent(int id)
{
    var student = await _context.Students.FindAsync(id);
    if (student == null)
    {
        return NotFound();
    }
    _context.Students.Remove(student);
    await _context.SaveChangesAsync();
    return NoContent();
}
private bool StudentExists(int id)
{
    return _context.Students.Any(e => e.ID == id);
}
}
```

Output:







b. Create an API endpoint to calculate Factorial of a Number and test it using Postman.

Code:

MathController.cs

using Microsoft.AspNetCore.Mvc;

namespace FactorialAPI.Controllers

{

[Route("api/[controller]")]

[ApiController]

public class MathController : ControllerBase

{

[HttpGet("factorial/{number}")]

public IActionResult GetFactorial(int number)

{

if (number < 0)

{

return BadRequest("Factorial not possible for negative numbers.");

}

long factorial = 1;

for (int i = 1; i <= number; i++)

{

factorial *= i;

}

return Ok(new

```
{  
    Number = number,  
    Factorial = factorial  
});  
}  
}
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services
```

```
builder.Services.AddControllers();
```

```
var app = builder.Build();
```

```
// Configure middleware
```

```
app.UseHttpsRedirection();
```

```
app.UseAuthorization();
```

```
// Map controller endpoints
```

```
app.MapControllers();
```

```
app.Run();
```

Output:

